BREVET DE TECHNICIEN SUPERIEUR CIEL
SESSION 2026

Dossier technique du projet
Partie individuelle

Projet : **Acces Campus**

Etudiant : **Emile Guillerm**

Etudiant 3 - Specialite IR

Lycée Saint Aubin La Salle - Verrieres en Anjou

Perimetres de responsabilite :

Base de Données (BDD) + API REST (PSW) + Infrastructure réseau externe

Table des matieres

Table of Contents

1 - Situation dans le projet 2

1.1 - Synoptique de la realisation 2
1.2 - Description de la partie personnelle 3
1.2.1 - Objectifs de ma realisation 3

2 - Realisation : Conception et deploiement de la base de donnees 4

2.1 - Justification des choix technologiques 4
2.1.1 - Systeme d'exploitation : Debian 12 4
2.1.2 - SGBD : PostgreSQL 4
2.1.3 - Langage de programmation : C++ 4
2.1.4 - Specifications materielles 4
2.2 - Conception detaillee 5
2.2.1 - Modele Conceptuel de Donnees (MCD) 5

3 - Realisation : Developpement de l'API REST 6

3.1 - Justification du choix : Framework Drogon 6
3.2 - Architecture logicielle de l'API 6
3.3 - Documentation des routes API 6
3.3.1 - Routes Utilisateurs 6
3.3.2 - Routes Badges 6
3.3.3 - Routes Salles et Reservations 7
3.3.5 - Format des echanges JSON 7
3.4 - Diagramme de sequence 7

1 - Situation dans le projet

1.1 - Synoptique de la réalisation

Le synoptique ci-dessous présente l'architecture matérielle et logicielle du système Accès Campus telle qu'elle est déployée en phase de développement.

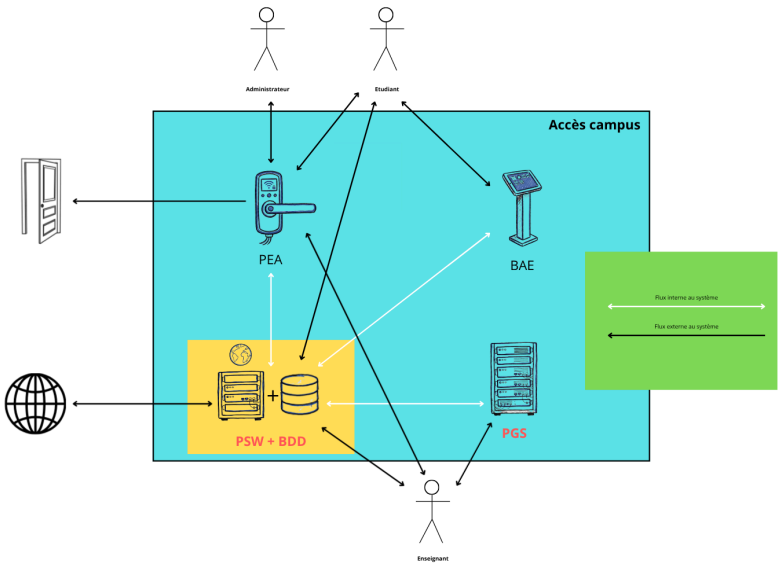
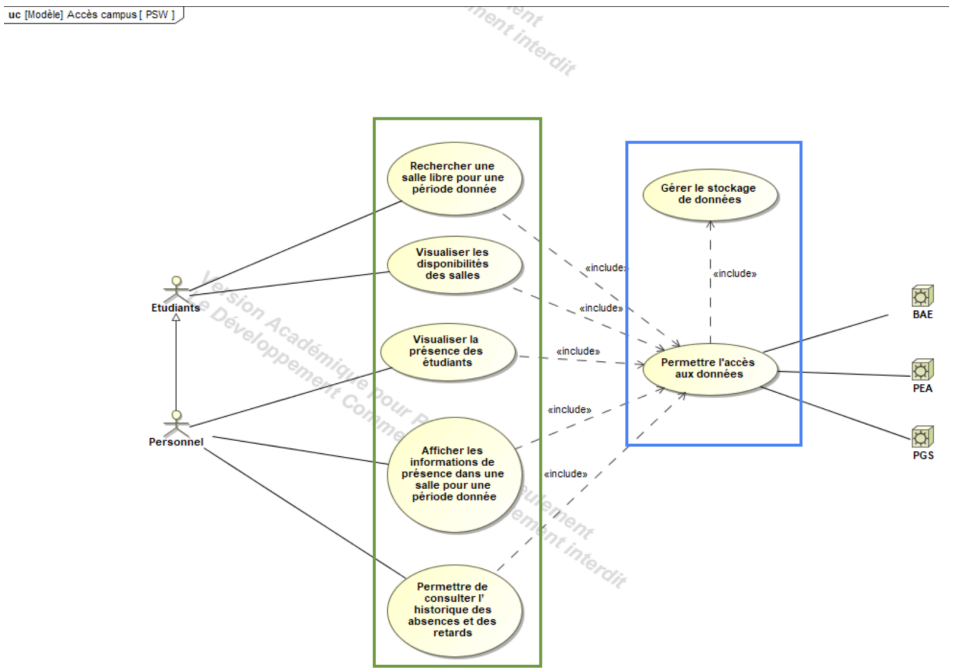


Figure 1 : Synoptique de la réalisation en phase de développement

Le serveur PSW+BDD tourne sur une machine avec Proxmox sous Debian (CLI). En développement, le serveur et la base de données cohabitent sur la même machine physique, alors qu'en production on pourrait envisager une séparation sur deux machines distinctes pour des raisons de performances et de sécurité.

2 - Réalisation : Conception et déploiement de la base de données



2.1 - Justification des choix technologiques

2.1.1 - Systeme d'exploitation : Debian 12

Pour le serveur hébergeant la base de données et l'API REST, j'ai choisi Debian 12 en mode CLI (sans interface graphique). Ce choix se justifie par la stabilité reconnue de Debian pour les environnements serveurs, sa faible consommation de ressources, et le fait qu'il s'agit d'un OS open source et gratuit. Debian en CLI économise les ressources graphiques et réduit les risques de crash. Les mises à jour sont contrôlées par l'administrateur, évitant les redémarrages imprévus qui seraient critiques pour un serveur de production.

2.1.2 - SGBD : PostgreSQL

Mon choix se portera sur PostgreSQL:

- PostgreSQL renvoie une erreur en cas de tentative d'insertion
- PostgreSQL gère les connexions asynchrones et le multi-processus, ce qui est important pour les pics de charge (début des cours)
- PostgreSQL est open source, gratuit et tres bien documente

Le processus PostgreSQL tourne en tant que service indépendant sur le serveur Debian. Le programme principal (API REST Drogon) communique avec lui via le protocole PostgreSQL (port 5432) en local.

2.1.3 - Langage de programmation : C++

J'ai choisi le C++ pour le développement de l'API REST. Ce choix de langage bas niveau offre des performances maximales et une maîtrise fine des ressources système, ce qui est adapté à un serveur devant gérer des pics de connexions simultanées (jusqu'à 2000 élèves et 300 salles).

2.1.4 - Specifications materielles

Les spécifications minimales retenues pour le serveur, basées sur le double des exigences Debian :

Composant	Specification
Processeur	2 coeurs minimum
RAM	4 Go
Stockage	32 Go (pas d'images/videos a stocker)
Reseau	Port Ethernet

2.2 - Conception detaillee

2.2.1 - Modèle Logique de Données (MLD)

Le MLD ci-dessous modélise les entités du système et leurs associations.

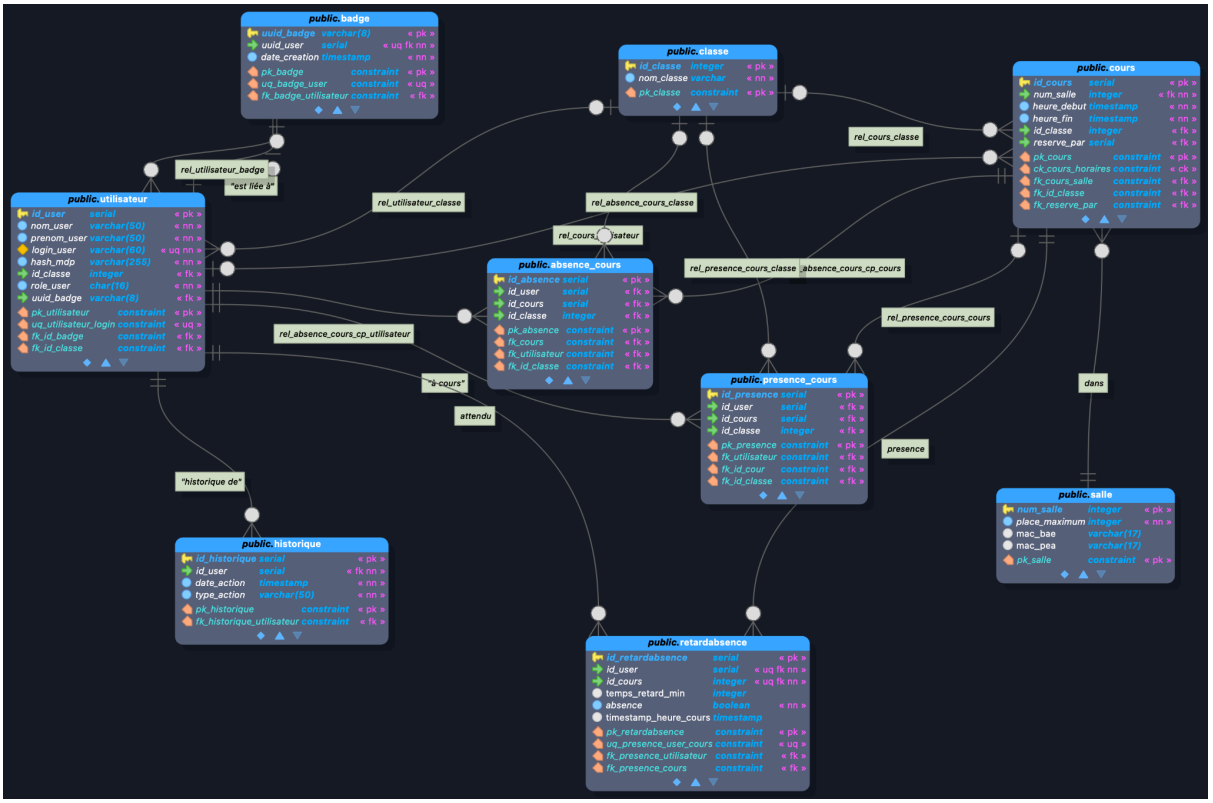


Figure 3 : Modèle Logique de Données (MLD)

3 - Réalisation : Développement de l'API REST

3.1 — Définition : qu'est-ce qu'une API REST ?

Une API (Application Programming Interface) est une interface logicielle qui définit un contrat de communication entre deux composants:

Le style architectural REST (Representational State Transfer) impose plusieurs contraintes à cette interface :

- Sans état (*stateless*) : chaque requête HTTP est indépendante.
- Interface uniforme : les opérations CRUD sont mappées sur les verbes HTTP : `GET` (lire), `POST` (créer), `PUT` (modifier), `DELETE` (supprimer).
- Représentation des ressources : les données échangées sont sérialisées dans un format standard. Le format que j'utilise est JSON (JavaScript Object Notation), lisible par tous les langages utilisés dans le projet.

3.2 — Le framework Drogon

Drogon est un framework HTTP C++ asynchrone et non-bloquant, basé sur des boucles événementielles (*event loop*).

Adéquation aux exigences de performance. Drogon utilise plusieurs threads en parallèle, chacun exécutant sa propre boucle événementielle. Cela permet de traiter plusieurs requêtes HTTP simultanément sans bloquer. Support natif de PostgreSQL. Drogon intègre un connecteur PostgreSQL asynchrone et un ORM (Object-Relational Mapping) généré automatiquement par l'outil `drogon_ctl`. Les classes C++ représentant les entités de la

base (`Badge`, `Utilisateur`, `Salle`, etc.) ont été générées directement depuis le schéma PostgreSQL.

Support des coroutines C++20. L'interface `CoroMapper` de Drogon permet d'écrire les accès à la base de données sous forme de coroutines C++20, avec les mots-clés `co_await` et `co_return`.

Routage HTTP intégré. Drogon propose un système de routage déclaratif permettant de définir les endpoints REST directement dans le code des contrôleurs, via des macros de chemin. Cela sera développé dans la section suivante.

ORM: L'outil drogon_ctl create model se connecte à PostgreSQL, inspecte le schéma des tables, et génère automatiquement des classes C++ représentant chaque table. Par exemple, la table badge donne lieu à une classe Badge avec des getters/setters pour chaque colonne (getBadgeid(), getStatutbadge(), etc.).

3.3 — Difficultés liées à l'utilisation de Drogon avec C++20

Drogon exploite des fonctionnalités récentes du standard C++ qui ont introduit des difficultés techniques pendant le développement.

Les coroutines C++20, une syntaxe complexe à maîtriser. Les coroutines sont une fonctionnalité introduite dans C++20 permettant d'écrire du code asynchrone de façon séquentielle. Un *point de suspension* (`co_await`) permet à la coroutine de rendre la main au thread sans bloquer, puis d'être reprise automatiquement à la réception du résultat.

Avant d'utiliser les coroutines, la première version du code utilisait les callback: chaque opération asynchrone prenait en paramètre une lambda capturant le résultat. L'imbrication de plusieurs appels successifs produisait un code imbriqué, difficile à lire, à déboguer et à maintenir.

Les coroutines C++20 nécessitent que le compilateur soit explicitement configuré avec le flag `-std=c++20`.

3.4 — Définition des routes API avec Drogon

Dans Drogon, les routes HTTP sont définies directement dans les fichiers d'en-tête des contrôleurs via le système de macros `PATH_ADD` et `METHOD_ADD`, associé à la classe de base `HttpController`.

Principe de routage. Lors du démarrage, Drogon inspecte tous les contrôleurs enregistrés et construit une table de routage interne. À réception d'une requête HTTP, il extrait le verbe et l'URI, recherche une correspondance dans la table, instancie le contrôleur approprié et invoque la méthode associée.

J’ai organisé les contrôleur par entité métier plutôt que par sous-système : un contrôleur `BadgeController` gère toutes les opérations sur les badges (lecture, ajout, modification, suppression), un `SalleController` gère les salles et réservations, etc. Cette organisation suit le principe REST qui veut que chaque URI identifie une ressource, indépendamment de l'opération.

4 - Exemple : Requête planning salle (PSW)

4.1 — Diagramme de séquence

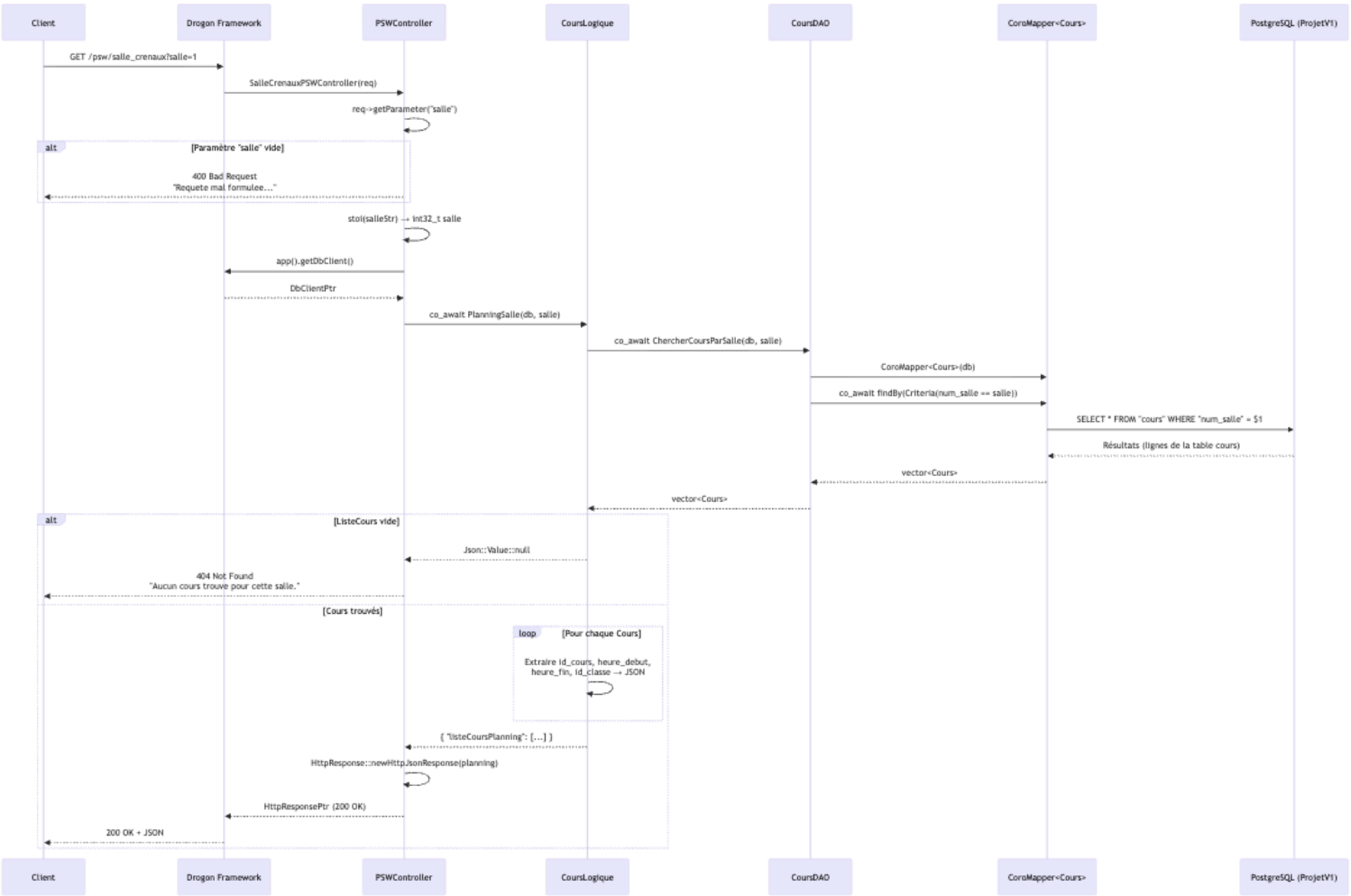


Figure 4 : Diagramme Séquence

4.2 — Postman pour envoyer des requêtes

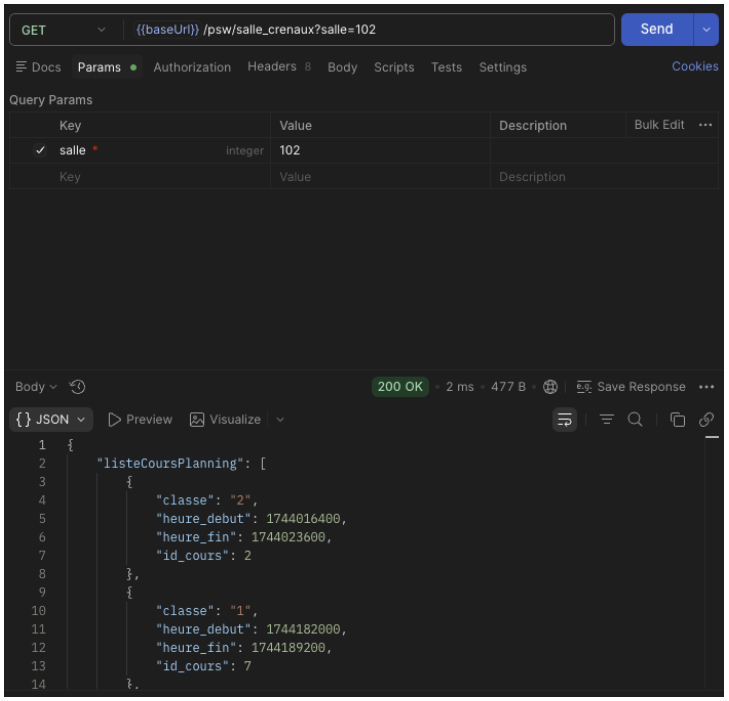


Figure 5 : Requête GET avec postman

Console du programme:

20260408 00:34:08.071242 GET /psw/salle_crenaux?salle=102 [0] (127.0.0.1:51482 - 127.0.0.1:80) 200 OK 330 0.017468

Confirmation (code 200) de la bonne exécution de la requête avec le résultat sous format JSON avec la liste de salle.

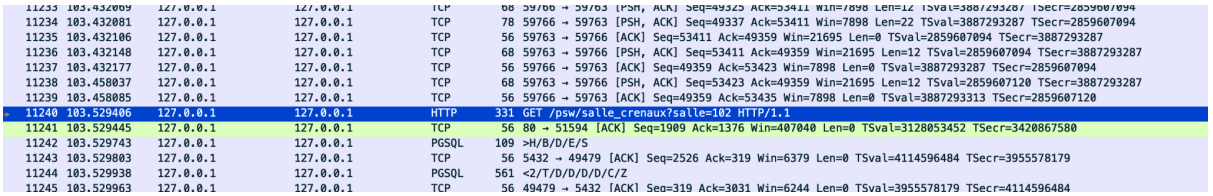


Figure 6 : Analyse du réseau avec wireshark

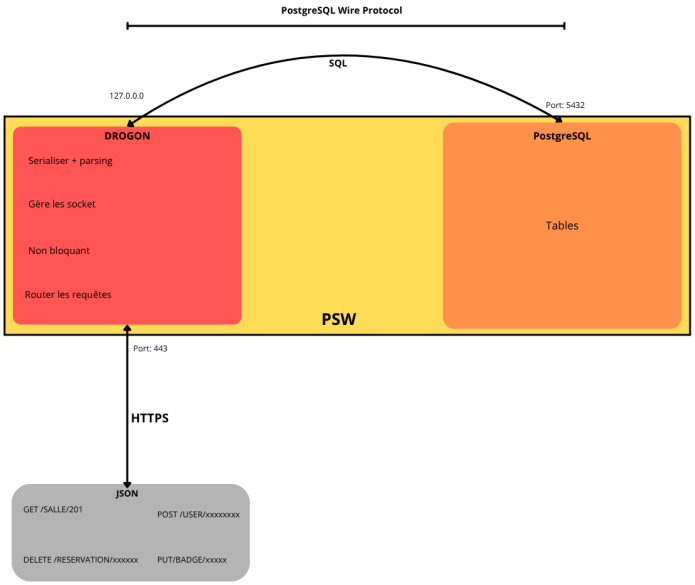
5 - Teste unitaire

1.1 – Environnement de test

Element	Description
Systeme d’exploitation	macOS (developpement)
SGBD	PostgreSQL
Framework	Drogon

Compilateur	C++20
IDE	CLion sur Mac M4
Outil de test API	Postman Desktop
Base de donnees	9 tables : utilisateur, badge, salle, cours, classe, presence_cours, absence_cours, retardabsence, historique

Element teste	GET /psw/salle_crenaux et GET /psw/etat_salle
Objectif du test	Vérifier que l'API retourne correctement le planning des salles avec les créneaux réservés et libres.
Nom du testeur	Emile Guillerm
Date	Avril 2026
Moyens mis en oeuvre	Postman, serveur Drogon démarrer, PostgreSQL avec données de test
Preconditions	Salle 101 reservee le 07/04/2026 de 08h a 10h. Salle 102 libre. Serveur PSW demarre.
Procedure du test	1. Envoyer GET {{base_url}}/psw/etat_salle. 2. Verifier le code HTTP 200 et le format JSON. 3. Verifier que la salle 101 apparait comme occupee et la 102 comme libre. 4. Envoyer GET {{base_url}}/psw/salle_crenaux?num_salle=101. 5. Verifier que le creneau 08h-10h apparait dans le planning.

Description du vecteur de test	Resultat attendu	Resultat obtenu	Validé
GET /psw/etat_salle → liste toutes les salles	HTTP 200, JSON array avec statut par salle	200 OK, JSON conforme	**OK**
GET /psw/salle_crenaux?num_salle=101	HTTP 200, planning avec creneau 08h-10h	Creneau affiche	**OK**
GET /psw/etat_salle sans donnees en base	HTTP 500	erreur	**NON**

Conclusion: Schéma de représentation du système